

Dla ucznia: jak pisać gry na konsolę

Masz przed sobą małą konsolę do gier i jej **emulator** – program, który udaje konsolę na komputerze. Gry piszesz w języku **C++**, w tym samym, w którym pisze się prawdziwe gry na Nintendo czy PlayStation. Każda gra, którą napiszesz, będzie działać w emulatorze, a potem na prawdziwej konsoli z ekranem dotykowym.

Jak zacząć

1. Kliknij skrót **Lekcje C++** na pulpicie. Otworzy się VS Code – program do pisania kodu.
2. Po lewej stronie znajdź folder `src` → `games` → `lekcje` → `01_wizytowka` i kliknij `gra.cpp`.
3. Wciśnij **F6** (albo Ctrl+Shift+B). Komputer przez chwilę kompiluje (zamienia kod na program), potem otwiera okno z grą.
4. Zamykasz okno klawiszem **Esc**. Zmieniasz coś w kodzie, znów F6. I tak w kółko – to całe programowanie.

Obok każdego `gra.cpp` jest `README.md` – tam jest lekcja: co nowego, jak to działa i zadania. Czytaj README, zmieniaj `gra.cpp`, sprawdzaj F6.

Klawisze w emulatorze

Konsola ma krzyżak i przyciski jak pad. W emulatorze udają je klawisze:

klawisz konsoli	na klawiaturze	do czego zwykle
góra / dół / lewo / prawo	strzałki	ruch
A	Z	skok, zatwierdź
B	X	akcja, bieg
X	S	dodatkowy
Y	A	dodatkowy
START	Enter	pauza (należy do konsoli, gra jej nie widzi)
–	Esc	zamknij emulator

Lewy przycisk myszy = palec na ekranie dotykowym. Prostokąty na dole ekranu to dotykowy pad – znikają, gdy użyjesz klawiszy.

Jak wygląda gra

```
#include "console/console.h"    // funkcje do rysowania i sterowania

namespace {                    // "pudelko" na twoja gre - nie ruszaj

int x = 100;                   // twoje zmienne

void setup()                   // RAZ, na starcie
{
    x = 100;
}

void frame()                   // 60 razy na sekunde
{
    clear(BLACK);              // wyczyszc ekran
    if (held(RIGHT)) {         // jesli trzymasz strzalke w prawo...
        x = x + 2;             // ...przesun
    }
    rect(x, 100, 20, 20, RED); // narysuj kwadrat
}

} // namespace                // koniec pudelka - nie ruszaj

CONSOLE_ADD_GAME(moja, "Moja gra", "Opis w menu") // rejestracja
```

Ekran ma **800 × 480** punktów (pikseli). `(0, 0)` to lewy górny róg. `x` rośnie w prawo, `y` rośnie **w dół**.

Ściągawka: co możesz wywołać

Pełny opis każdej funkcji z przykładami i plakatem: [API.md](#) (także jako `docs/pdf/API.pdf`). W menu konsoli jest też gra **Plakat API** – pokazuje wszystkie figury na jednym ekranie z podpisami.

co	jak
wyczyść ekran	<code>clear(BLACK)</code>
prostokąt	<code>rect(x, y, szerokosc, wysokosc, KOLOR)</code>
sam obrys	<code>rect_outline(x, y, w, h, KOLOR)</code>
koło	<code>circle(srodek_x, srodek_y, promien, KOLOR)</code>
linia	<code>line(x1, y1, x2, y2, KOLOR)</code>
napis	<code>text(x, y, "Tekst", KOLOR, rozmiar 1-4)</code> – bez polskich liter
liczba na ekranie	<code>text(x, y, wynik, KOLOR, 3)</code>
kolory	BLACK WHITE GRAY RED GREEN BLUE YELLOW ORANGE PURPLE BROWN SKY , ciemne: DARK_RED DARK_BLUE ... , własny: <code>rgb(255, 0, 128)</code>
klawisz trzymany	<code>held(LEFT)</code> – UP DOWN LEFT RIGHT A B X Y
klawisz właśnie wciśnięty	<code>pressed(A)</code>
losowa liczba od 1 do 6	<code>random(1, 6)</code>
przycięcie do zakresu	<code>clamp(x, 0, 760)</code>
czy prostokąty się dotykają	<code>overlaps(x1, y1, w1, h1, x2, y2, w2, h2)</code>
szerokość / wysokość ekranu	<code>screen_width()</code> / <code>screen_height()</code>
numer klatki, sekundy	<code>frame_count()</code> , <code>seconds()</code>
obrazek z liter	<code>Sprite s = load_sprite(rysunek);</code> potem <code>sprite(s, x, y, false, 3)</code> – 3 = powiększenie (lekcja 10)
obrazek z pliku PNG	<code>Sprite s = load_image("hero.png");</code> – plik w <code>assets/<id gry>/</code> , przezroczyste tło działa (lekcja 10)
mapa z liter	<code>load_map(MAPA, "#=B")</code> , <code>map_tile(col, row)</code> , <code>map_set(...)</code> , <code>draw_tiles('#'</code> , <code>KOLOR</code> , <code>cam_x</code>) (lekcja 12)
ruch z kolizjami z mapą	<code>MoveResult r = move_box(x, y, w, h, vx, vy);</code> → <code>r.on_ground</code> (lekcja 12)
zaczynij grę od nowa	<code>restart()</code>
pokaż zmienną w rogu	<code>watch("x", x)</code>
napisz w terminalu	<code>print("tekst", liczba)</code>

Jak czytać błąd

Gdy coś jest nie tak, po F6 zamiast gry pojawi się w terminalu czerwony tekst, na przykład:

```
src/games/lekcje/02_pilka/gra.cpp:14:5: error: 'predkosc' was not declared in this scope
```

Czytaj to tak: **plik : linia 14 : znak 5** : błąd: „`predkosc` nie zostało zadeklarowane” – czyli użyłeś nazwy, której komputer nie zna (literówka? zapomniałeś `int predkosc = 3; ?`). Kliknij w tę linię w terminalu – VS Code przeniesie cię w to miejsce w kodzie. Najczęstsze komunikaty:

komunikat	co znaczy
<code>expected ';' before ...</code>	brakuje średnika na końcu poprzedniej linii
<code>'xyz' was not declared in this scope</code>	nieznana nazwa: literówka albo brak deklaracji
<code>expected '}' at end of input</code>	brakuje zamykającej klamry <code>}</code>
<code>no matching function for call to 'rect(...)'</code>	zła liczba lub typ argumentów – sprawdź w ściągawce
<code>redefinition of 'x'</code>	tę nazwę już zadeklarowałeś wyżej
<code>cannot convert 'Color' to 'int'</code>	pomyliłeś kolejność: kolor jest ostatni
<code>multiple definition of 'setup()'</code>	w jakiejś lekcji brakuje <code>namespace {</code> na górze
Permission denied / collect2: error przy linkowaniu	emulator jest jeszcze otwarty – zamknij okno (Esc) i F6

Komputer pokazuje najwyżej 3 błędy naraz. Napraw pierwszy – kolejne często znikają same.

Podglądanie, co się dzieje

- `watch("nazwa", zmienna)` w `frame()` pokazuje wartość w prawym górnym rogu ekranu, na żywo. Idealne do sprawdzania „dlaczego piłka znika”.
- `print("tekst", liczba)` pisze w terminalu pod oknem. Uwaga: w `frame()` to 60 linii na sekundę – lepiej w `if`.
- Zadanie **LEKCJA: Ślad** (Ctrl+Shift+P → „Run Task”) uruchamia grę na 5 sekund bez klawiszy i wypisuje wartości z `watch` co pół sekundy.
- Zadanie **LEKCJA: Zrzut ekranu** zapisuje obrazek `zrzut.bmp` w katalogu lekcji.
- Zadanie **LEKCJA: Sprawdź zadania** uruchamia testy – powie ci, które zadania z README są zrobione dobrze.

Zapisywanie postępu (git)

Twój kod jest w **repozytorium git** – to jak punkty zapisu w grze. Po skończonym zadaniu:

1. Kliknij ikonę **Source Control** po lewej (trzy kółka połączone liniami) albo Ctrl+Shift+G.
2. W polu na górze wpisz, co zrobiłeś, np. `Lekcja 02 zadanie 3 - pilka zmienia kolor`.
3. Kliknij ✓ **Commit**. Gotowe – punkt zapisu.

Wysłaniem do internetu (push) zajmuje się rodzic.

Zasady

- Zmieniasz pliki **tylko** w `src/games/lekcje/`. Reszta to silnik konsoli – jeśli chcesz coś w nim zmienić, powiedz.
- Zmienne nazywaj **małymi literami** po angielsku: `ball_x`, `score`, `speed`. Stałe WIELKIMI: `SIZE`.
- W kodzie **nie używaj polskich znaków** (ą, ę, ł...) – ani w napisach, ani w komentarzach. Czcionka konsoli ich nie ma.
- Nie usuwaj linii `namespace { i } // namespace` – bez nich dwie lekcje pokłóćą się o nazwy.
- Rozwiązania zadań są w katalogu `rozwiązania/` każdej lekcji. Najpierw spróbuj sam przez 15 minut – dopiero potem zaglądaj.

Gdy coś nie działa

objaw	co zrobić
F6 nic nie robi	kliknij najpierw w plik <code>gra.cpp</code> – zadanie uruchamia grę z otwartego pliku
nie ma gry "..."	nazwa w <code>CONSOLE_ADD_GAME(...)</code> w <code>gra.cpp</code> i w <code>lista.h</code> muszą być takie same
gra się nie zmieniła po edycji	plik niezapisany? (biała kropka na karcie); VS Code zapisuje sam po chwili
błąd <code>collect2 / Permission denied</code>	okno emulatora jest otwarte – zamknij Esc
czerwone podkreślenia w edytorze, a kompiluje się	to tylko podpowiedzi; jeśli VS Code zapyta o „IntelliSense provider”, wybierz Yes
Windows „chronił komputer” przy starcie <code>console_sim.exe</code>	kliknij „Więcej informacji” → „Uruchom mimo to”; to twój własny, świeżo skompilowany program

Lekcje

nr	katalog	czego się uczysz	gra
00	00_szablon	budowa pliku gry	napis
01	01_wizytowka	zmienne, stałe, współrzędne	wizytówka
02	02_pilka	if	odbijająca się piłka
03	03_lapacz	klawisze, && \ \ else , losowanie	łapanie piłek
04	04_pong	funkcje	Pong
05	05_inwazja	pętle for	ściana kosmitów
06	06_inwazja_strzal	tablice	Space Invaders
07	07_waz	tablice jako listy	Snake
08	08_flappy	struktury	Flappy
09	09_breakout	tablice struktur	Breakout
10	10_bohater	własna grafika	zbieracz monet
11	11_flappy_pelny	stan gry, HUD, rekord	Flappy kompletny
12	12_mini_mario	mapa kafelków, skok	mini-platformówka

| 13 | 13_twoja_gra | własny projekt od pomysłu do gry | twoja |

Rób je po kolei – każda korzysta z poprzednich.